

Due Date: Week 4

Project ID & Title: #5 — Healthcare Clinic & Telemedicine Portal

A. Project Identity

Team Name: Phải Tin tui

Team Members:

- Nguyễn Hoàng Dũng — n24dccn110@student.ptithcm.edu.vn
- Lý Thành Đạt — n24dccn103@student.ptithcm.edu.vn
- Phạm Trần Quốc Việt — n24dccn182@student.ptithcm.edu.vn

Project Title: Centralized Telemedicine Portal and Clinical Management Database System

B. Report Structure

1. Introduction & Project Scope

(Adapted from ISO/IEC/IEEE 29148)

1.0 Problem Statement

Small clinics that still run on paper records and phone calls face a few recurring problems: doctors get double-booked, patient histories are incomplete or scattered across folders, and prescriptions are hard to track. Patients have no way to check a doctor's availability before calling, and staff have no shared record that links an appointment to the diagnosis and prescription that came out of it. This project builds a relational database to fix that — one schema covering patient profiles, doctor schedules, appointment booking, medical records, and prescriptions, with constraints that prevent bad data from entering in the first place.

1.1 System Objective

This project designs a relational database for a small clinic that handles both walk-in visits and remote consultations. The database must store patient profiles, doctor shift schedules, appointment bookings, medical records, and prescriptions in one place — with no duplicate data and no orphaned records. The target is to replace the current paper-and-phone workflow with a schema that a backend application can query directly.

1.1.1 Actor Definitions (Scope & Business Context)

The actors below are defined in the specific context of this project, not as generic roles, to avoid ambiguity in access-control design:

- **Patient (Outpatient only):** An individual who visits the clinic for a scheduled consultation - in-person or telemedicine - and leaves after the appointment. This project models outpatient care exclusively; inpatient admission, ward/bed assignment, and multi-day hospitalization are out of scope, so PATIENT has no admission or discharge attributes.
- **Doctor - Treating Doctor (GP / Specialist):** A clinician directly responsible for a patient's appointment, diagnosis, and prescription. The schema distinguishes doctors by clinical role (GP vs. Specialist, BR-01), since that determines room assignment (BR-03) and specialty data (BR-02). Employment type (Full-time vs. Part-time, see below) is tracked separately and affects compensation only.
- **Doctor - Employment Type (Compensation Basis):** Every doctor is additionally classified as FULL_TIME (fixed monthly salary, independent of shift count) or PART_TIME (compensated per hour actually worked, per SHIFT, at a registered rate). This classification is independent of both the GP/Specialist axis (BR-01) and the Treating/Consulting axis (BR-21 onward) - a doctor can be any combination of the three, e.g. Specialist + Full-time + occasionally

Consulting. Employment type affects compensation only: it does NOT change record-access permissions - both Full-time and Part-time doctors are governed identically by BR-21/BR-22 (access limited to cases where they are the assigned treating doctor, unless invited as a consulting doctor).

- **Doctor - Consulting Doctor (per-case role):** Any registered doctor (GP or Specialist) can additionally act as a Consulting Doctor on a specific case: invited by the treating doctor to review one particular medical record and give a professional opinion, without seeing the patient directly, prescribing, or editing the record. This is not a separate DOCTOR subtype - it is a temporary, per-case relationship represented by the CONSULTATION_REQUEST entity (Section 2.1.2) and governed by BR-21 to BR-23.
- **Admin / Receptionist:** Administrative staff who never provide clinical care. Their scope is limited to doctor account creation, shift/room scheduling infrastructure, the medication catalog, and staff permission management (BR-18 to BR-20); they have no default access to MEDICAL_RECORD or PRESCRIPTION content.

1.1.2 Functional Requirements (Actor-Action-Data Format)

Patient

1. The system must allow a Patient to register and update their demographic and contact profile.
2. The system must allow a Patient to view a Doctor's real-time available time slots before booking.
3. The system must allow a Patient to book an in-person or telemedicine appointment against a confirmed Doctor shift.
4. The system must allow a Patient to view their own appointment history, medical records, and prescriptions.

Doctor

5. The system must allow a Doctor to register their working shifts across designated dates and time slots.
6. The system must allow a Doctor to view and update the status of appointments assigned to them (e.g., CONFIRMED → IN_PROGRESS → COMPLETED).
7. The system must allow a Doctor to record clinical symptoms, diagnosis, and notes for a completed appointment.
8. The system must allow a Doctor to issue a digital prescription containing one or more medication line items for a medical record.

The system must allow a treating Doctor to create and manage a CONSULTATION_REQUEST for one specific medical record, including the invited consulting doctor, reason, status, and response timestamp.

The system must allow an invited consulting Doctor to review the specified medical record only while the request is APPROVED, in read-only mode, without editing the record or issuing a prescription.

Admin / Receptionist

9. The system must allow an Admin to create and manage Doctor accounts, including specialization type (GP or Specialist).
10. The system must allow an Admin to maintain the clinic's medication catalog (add, update, deactivate items).
11. The system must allow an Admin to assign role-based access permissions to staff accounts.

1.1.3 Non-Functional Requirements (Database Constraints)

- **NFR-01** — Data Integrity: Unique indexes and composite constraints must prevent any two appointments from occupying the same doctor slot at the same time.
- **NFR-02** — Access Control: Role-Based Access Control (RBAC) must be applied at the database level. A patient's medical record is readable only by that patient and the doctors or admin staff who are assigned to their case.
- **NFR-03** — Performance: Common read queries — doctor directory, available time slots, medication catalog — must return results in under 200 ms on a dataset of up to 10,000 patients and 50 active doctors. This will be verified with EXPLAIN ANALYZE in Phase 3.
- **NFR-04** — Security: Columns storing diagnosis text, prescription details, and vital signs must be encrypted at rest (AES-256 at the storage level). Every INSERT, UPDATE, and SELECT against the MEDICAL_RECORD table must create an immutable entry in MEDICAL_RECORD_AUDIT_LOG, recording the acting user (patient, doctor, or admin), the action, and the timestamp.

1.2 Business Rules & Constraints

Patient

- **BR-00:** Each patient is uniquely identified in the system (patient_id) and must have at least one verifiable contact method (phone or email) on file.

Doctor Specialization (EER Hierarchy Requirement)

- **BR-01:** Every doctor must be classified as either a General Practitioner (GP) or a Specialist before the account is activated. The specialization is total — a doctor with no subtype cannot be assigned appointments — and disjoint — a doctor cannot hold both subtypes at once. This is enforced at registration; the admin cannot save a doctor record without selecting one type.
- **BR-02:** Each Specialist is associated with a specific medical specialty (e.g., Cardiology, Dermatology, Pediatrics) and must record documented years of practice.
- **BR-03:** Each GP is assigned to a specific consultation room (stored in a ROOM entity) for each shift. No two GPs may be assigned to the same room during overlapping shift times. This constraint will be enforced via a trigger in Phase 3, similar to BR-09.

BR-26: ROOM assignment is modeled only for GP shifts. IN_PERSON appointments with Specialists are supported, but the physical room assignment for Specialist consultations is outside this database's scope and is managed by the clinic's external scheduling/front-desk process.

- **BR-24:** Every DOCTOR has an employment_type ('FULL_TIME' or 'PART_TIME'), a mandatory attribute independent of spec_type (BR-01). If employment_type = 'PART_TIME', every SHIFT belonging to that doctor must have a rate_per_hour (DECIMAL, > 0) — the rate agreed for that specific shift, fixed at scheduling time based on the doctor's seniority and the nature of the shift (regular, evening, weekend, etc.). The rate is stored on SHIFT rather than DOCTOR so that the historical pay of completed shifts is preserved even if the general rate card changes later. If employment_type = 'FULL_TIME', rate_per_hour is NULL, since salary is fixed monthly and managed outside this system's scope. This rule affects compensation calculation only and does NOT alter BR-21/BR-22 — medical-record access scope is identical for Full-time and Part-time doctors.

Doctor Shift Schedules

- **BR-04:** A doctor registers multiple working shift entries. Each shift entry belongs to exactly one doctor.
- **BR-05:** A doctor cannot have overlapping working shift periods on the same calendar date.

Patient Appointments

- **BR-06:** A patient can book multiple appointments, but each appointment belongs to exactly one patient.
- **BR-07:** An appointment is assigned to exactly one doctor.
- **BR-08:** An appointment must be scheduled within an active working shift of the chosen doctor.
- **BR-09:** A doctor cannot have overlapping confirmed appointments (Strict Anti-Double Booking constraint). This rule is enforced at the implementation level via a database trigger or exclusion constraint in Phase 3 — it cannot be expressed as a simple cardinality on the EER diagram.
- **BR-10:** An appointment must be classified as either an in-person clinic visit or a remote telemedicine consultation.
- **BR-11:** The appointment lifecycle status must be constrained to: 'PENDING', 'CONFIRMED', 'IN_PROGRESS', 'COMPLETED', or 'CANCELLED'.

Medical Histories & Records

- **BR-12:** Each APPOINTMENT may produce at most one MEDICAL_RECORD, but only after the appointment status is set to 'COMPLETED'. A MEDICAL_RECORD must always reference a valid parent appointment (FK NOT NULL); it cannot exist on its own. In the EER diagram, the relationship from APPOINTMENT to MEDICAL_RECORD is (1 : 0..1) — one appointment, zero or one record — because pending or cancelled appointments never generate one.
- **BR-13:** A medical record stores vital signs, symptoms, clinical diagnosis, and notes recorded by the attending doctor.

Digital Prescriptions & Medications

- **BR-14:** A completed medical record can result in at most one digital prescription. A prescription cannot exist without an associated medical record.
- **BR-15:** A digital prescription can contain one or multiple medication line items.
- **BR-16:** A single medication can appear in many prescriptions, and one prescription can list many medications (M:N). This is resolved through an associative entity PRESCRIPTION_ITEM, which links PRESCRIPTION and MEDICATION and carries the per-item details: quantity, dosage, frequency, and duration.
- **BR-17:** Each PRESCRIPTION_ITEM must specify a positive quantity, exact dosage instructions, intake frequency, and treatment duration in days.
- **BR-19:** An Admin may add, update, or logically deactivate (is_active = FALSE) a medication entry in the MEDICATION catalog. A medication record referenced by an existing PRESCRIPTION_ITEM cannot be hard-deleted; deactivation is used instead to preserve historical prescription integrity.
- **BR-25:** All additions, modifications, or deactivations in the medication catalog must record an immutable entry in MEDICATION_AUDIT_LOG.

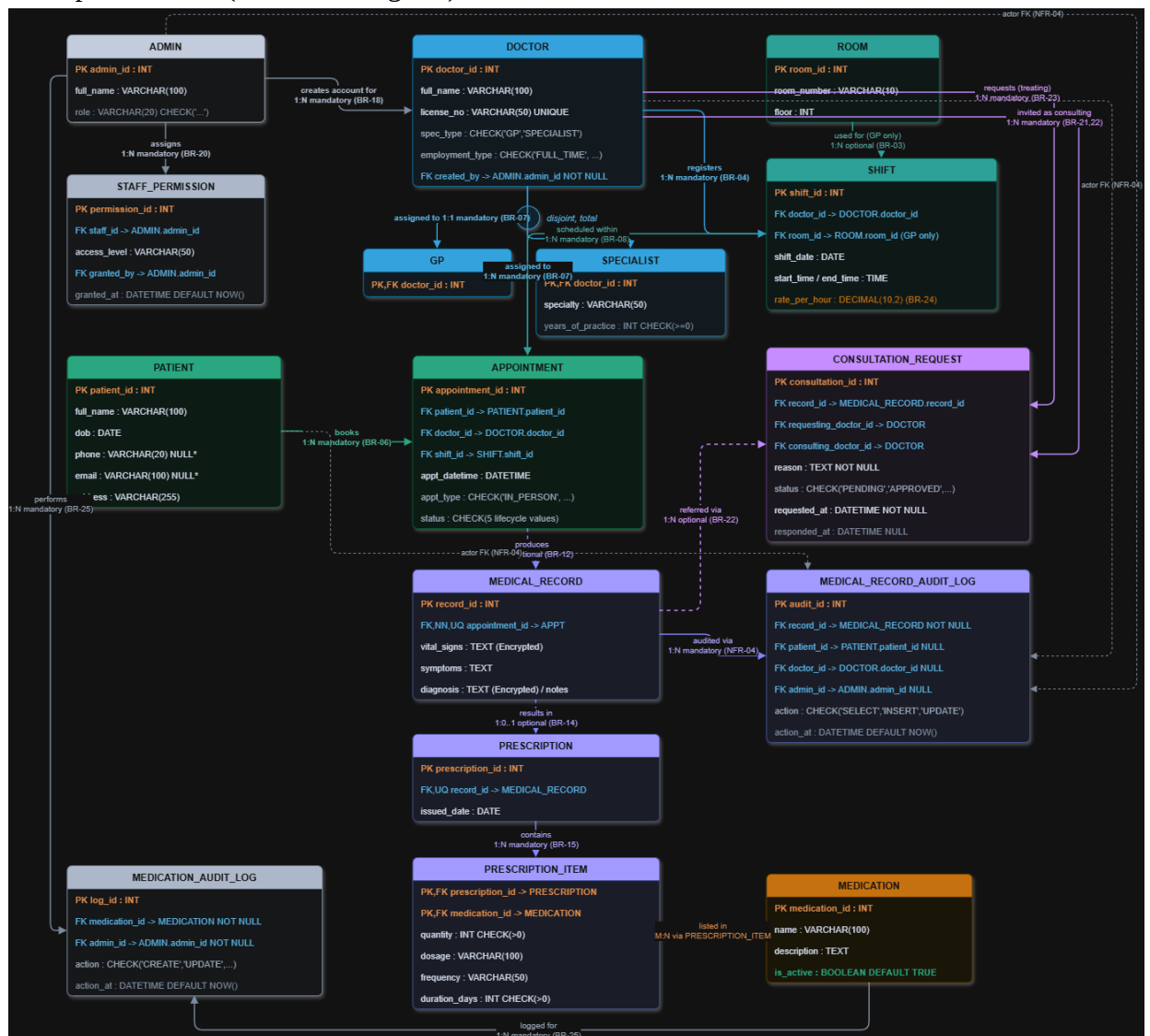
Administrative Governance & Consultation Access

- **BR-18:** An ADMIN account must exist in the system before any Doctor account can be created or activated. Each ADMIN is uniquely identified (admin_id) and must hold a designated role (e.g., 'RECEPTIONIST' or 'SYSTEM_ADMIN'). This is enforced by a NOT NULL FK reference from DOCTOR.created_by to ADMIN.admin_id.
- **BR-20:** Role-Based Access is managed through a STAFF_PERMISSION table linking ADMIN accounts to a set of permitted actions (e.g., 'CREATE_DOCTOR', 'MANAGE_MEDICATION', 'VIEW_ALL_RECORDS'). Each permission assignment records the admin who granted it and the timestamp, supporting the audit requirement in NFR-04. An admin cannot grant permissions exceeding their own role level.

- **BR-21:** By default, a doctor (GP or Specialist) may view the MEDICAL_RECORD and PRESCRIPTION of a patient only for appointments where that doctor is the assigned treating doctor (APPOINTMENT.doctor_id). A doctor cannot browse a patient's records from other doctors or specialties (e.g., a dentist cannot view a cardiology record of the same patient) merely because the patient is shared.
- **BR-22:** Cross-specialty or second-opinion access is granted only through a CONSULTATION_REQUEST: the treating doctor invites a named consulting doctor to review one specific medical record. The consulting doctor's access is read-only, scoped to that single record, and limited to the period the request remains 'APPROVED'; the consulting doctor cannot issue a prescription or edit the record.
- **BR-23:** Each CONSULTATION_REQUEST must record the requesting (treating) doctor, the invited consulting doctor, the target medical record, a reason, and a status ('PENDING', 'APPROVED', 'DECLINED', 'COMPLETED'), satisfying the audit trail required for access to sensitive medical data (NFR-04).

2. Database Design (ISO/IEC 19505 / IE Standards)

2.1 Conceptual Model (ER/EER Diagram)



2.1.1 Entity Definitions

Each entity in the EER diagram above represents the following business concept:

- **ADMIN:** A staff member with administrative privileges who manages doctor accounts, the medication catalog, and role-based access permissions (BR-18, BR-19, BR-20).
- **STAFF_PERMISSION:** A specific access permission granted to an admin account, recording which action is allowed and by whom (BR-20).
- **DOCTOR:** A clinician who registers shifts and is assigned to appointments. Every doctor is classified as either a GP or a Specialist (BR-01).
- **GP:** Subtype of DOCTOR representing a General Practitioner, who is assigned to a fixed consultation room per shift (BR-03).
- **SPECIALIST:** Subtype of DOCTOR representing a doctor with a declared medical specialty and years of practice (BR-02).
- **ROOM:** A physical consultation room used exclusively for GP shifts (BR-03).
- **SHIFT:** A block of time during which a doctor is available for appointments, on a specific date and time range (BR-04, BR-05).
- **PATIENT:** An individual who registers a profile in order to book appointments and view their own medical history (BR-00).
- **APPOINTMENT:** A booking made by a patient against a doctor's shift, either in-person or via telemedicine, tracked through a defined status lifecycle (BR-06 to BR-11).
- **MEDICAL_RECORD:** The clinical outcome of a completed appointment, capturing vital signs, symptoms, diagnosis, and doctor's notes (BR-12, BR-13).
- **PRESCRIPTION:** A digital prescription issued from a medical record (BR-14).
- **PRESCRIPTION_ITEM:** An associative entity resolving the M:N relationship between PRESCRIPTION and MEDICATION, carrying per-item dosage details (BR-15, BR-16, BR-17).
- **MEDICATION:** An item in the clinic's medication catalog, which can be deactivated but not deleted once referenced by a prescription (BR-19).
- **CONSULTATION_REQUEST:** A temporary, per-case request linking a treating doctor to an invited consulting doctor for read-only review of one specific medical record, with reason, status, and timestamps (BR-21 to BR-23).
- **MEDICATION_AUDIT_LOG:** A full history record of every CREATE, UPDATE, or DEACTIVATE action taken on a MEDICATION entry, recording which admin performed it and when (BR-25).

2.1.2 Relationship Definitions

Cardinality and participation for each relationship shown on the EER diagram:

Relationship	Cardinality	Description
ADMIN creates account for DOCTOR	1:N, mandatory on DOCTOR	Every doctor account must be created by exactly one admin; an admin can create many doctor accounts (BR-18).
ADMIN assigns STAFF_PERMISSION	1:N, mandatory on STAFF_PERMISSION	Each permission record is granted by exactly one admin; an admin can be granted many permissions (BR-20).
ADMIN performs MEDICATION_AUDIT_LOG	1:N, mandatory on MEDICATION_AUDIT_LOG	Every create, update, or deactivate action on the medication catalog must be executed by exactly one admin (BR-25).
DOCTOR ISA GP / SPECIALIST	Disjoint, total specialization	Every doctor must be classified as exactly one of GP or Specialist, never both, before activation (BR-01).
DOCTOR registers SHIFT	1:N, mandatory on SHIFT	Each shift belongs to exactly one doctor; a doctor can register many shifts (BR-04).
ROOM used for SHIFT (GP only)	1:N, optional on SHIFT	A room can be allocated for multiple GP shifts; only GP shifts reference a physical room, leaving the foreign key NULL for other shifts (BR-03).

Relationship	Cardinality	Description
DOCTOR assigned to APPOINTMENT	1:N, mandatory on APPOINTMENT	Each appointment is assigned to exactly one doctor; a doctor can handle multiple appointments (BR-07).
SHIFT scheduled within APPOINTMENT	1:N, mandatory on APPOINTMENT	Each appointment must be scheduled entirely within one active, confirmed working shift of the assigned doctor (BR-08).
PATIENT books APPOINTMENT	1:N, mandatory on APPOINTMENT	Each appointment is booked by exactly one patient; a patient can book multiple appointments (BR-06).
APPOINTMENT produces MEDICAL_RECORD	1:0..1, optional on APPOINTMENT	An appointment produces at most one medical record, and strictly after reaching COMPLETED status; a record cannot exist independently (BR-12).
MEDICAL_RECORD results in PRESCRIPTION	1:0..1, optional on MEDICAL_RECORD	A medical record results in at most one digital prescription; a prescription cannot exist without a valid medical record (BR-14).
PRESCRIPTION contains PRESCRIPTION_ITEM	1:N, mandatory on PRESCRIPTION_ITEM	A prescription contains one or more medication line items (BR-15).
PRESCRIPTION lists MEDICATION	M:N	A prescription can list many medications and a medication can appear across many prescriptions; resolved via the associative entity PRESCRIPTION_ITEM (BR-16).
MEDICATION logged for MEDICATION_AUDIT_LOG	1:N, mandatory on MEDICATION_AUDIT_LOG	Each audit log entry must reference exactly one target medication record (BR-25).
MEDICAL_RECORD referred via CONSULTATION_REQUEST	1:N, optional on MEDICAL_RECORD	A medical record may have zero or more consultation requests raised against it (BR-22).
DOCTOR (treating) requests CONSULTATION_REQUEST	1:N, mandatory on CONSULTATION_REQUEST	Each consultation request is initiated by exactly one treating doctor (BR-23).
DOCTOR (consulting) invited to CONSULTATION_REQUEST	1:N, mandatory on CONSULTATION_REQUEST	Each consultation request names exactly one consulting doctor, whose access is scoped to that request only (BR-21, BR-22).
MEDICAL_RECORD audited via MEDICAL_RECORD_AUDIT_LOG	1:N, mandatory on MEDICAL_RECORD_AUDIT_LOG	Every INSERT, UPDATE, or SELECT on a medical record creates one immutable audit entry recording exactly one actor (patient, doctor, or admin), the action, and timestamp (NFR-04).

2.2 Logical Schema Mapping (Elmasri-Navathe Algorithm)

Step 1: Regular Entity Types Mapping

- **ADMIN** (admin_id, full_name, role)
- **ROOM** (room_id, room_number, floor)
- **PATIENT** (patient_id, full_name, dob, phone, email, address)
- **MEDICATION** (medication_id, name, description, is_active)

MEDICAL_RECORD_AUDIT_LOG (audit_id, record_id, patient_id, doctor_id, admin_id, action, action_at)

Step 2: Specialization / Generalization Mapping (Disjoint, Total)

- Mapped using **Option 8A (Multiple Relations for Superclass and Subclasses)** to avoid excessive NULL attributes and ensure separate authorization constraints:
- **DOCTOR** (doctor_id, full_name, license_no, spec_type, employment_type, created_by)
- **GP** (doctor_id)
- **SPECIALIST** (doctor_id, specialty, years_of_practice)

Step 3: Binary 1:1 Relationship Mapping

- Embedded foreign keys with UNIQUE and NOT NULL constraints into the total participation entities:
- **MEDICAL_RECORD** (record_id, appointment_id, vital_signs, symptoms, diagnosis, notes)
- **PRESCRIPTION** (prescription_id, record_id, issued_date)

Step 4: Binary 1:N Relationship Mapping

- Propagated parent primary keys as foreign keys into the N-side relations:
- **STAFF_PERMISSION** (permission_id, staff_id, access_level, granted_by, granted_at)
- **SHIFT** (shift_id, doctor_id, room_id, shift_date, start_time, end_time, rate_per_hour)
- **APPOINTMENT** (appointment_id, patient_id, doctor_id, shift_id, appt_datetime, appt_type, status)
- **CONSULTATION_REQUEST** (consultation_id, record_id, requesting_doctor_id, consulting_doctor_id, reason, status, requested_at, responded_at)
- **MEDICATION_AUDIT_LOG** (log_id, medication_id, admin_id, action, action_at)

MEDICAL_RECORD_AUDIT_LOG (audit_id, record_id, patient_id, doctor_id, admin_id, action, action_at)

Step 5: Binary M:N Relationship Mapping

- Resolved into an associative relation carrying relationship attributes:
- **PRESCRIPTION_ITEM** (prescription_id, medication_id, quantity, dosage, frequency, duration_days)

2.2.1 Foreign Key & Referential Integrity Rules

This section defines the ON DELETE / ON UPDATE behavior for every foreign key in the schema. CASCADE is applied where the child entity cannot exist independently of its parent. RESTRICT is enforced to preserve historical, clinical, and audit integrity. SET NULL is used strictly where foreign keys are optional (SHIFT.room_id). ON UPDATE is CASCADE across all surrogate keys.

Table	FK Column	References	ON DELETE	ON UPDATE	BR
STAFF_PERMISSION	staff_id	ADMIN(admin_id)	CASCADE	CASCADE	BR-20
STAFF_PERMISSION	granted_by	ADMIN(admin_id)	RESTRICT	CASCADE	BR-20
DOCTOR	created_by	ADMIN(admin_id)	RESTRICT	CASCADE	BR-18
GP	doctor_id	DOCTOR(doctor_id)	CASCADE	CASCADE	BR-01
SPECIALIST	doctor_id	DOCTOR(doctor_id)	CASCADE	CASCADE	BR-01
SHIFT	doctor_id	DOCTOR(doctor_id)	CASCADE	CASCADE	BR-04
SHIFT	room_id	ROOM(room_id)	SET NULL	CASCADE	BR-03
APPOINTMENT	patient_id	PATIENT(patient_id)	RESTRICT	CASCADE	BR-06
APPOINTMENT	doctor_id	DOCTOR(doctor_id)	RESTRICT	CASCADE	BR-07

Table	FK Column	References	ON DELETE	ON UPDATE	BR
APPOINTMENT	shift_id	SHIFT(shift_id)	RESTRICT	CASCADE	BR-08
MEDICAL_RECORD	appointment_id	APPOINTMENT(appointment_id)	RESTRICT	CASCADE	BR-12
CONSULTATION_REQUEST	record_id	MEDICAL_RECORD(record_id)	CASCADE	CASCADE	BR-22
CONSULTATION_REQUEST	requesting_doctor_id	DOCTOR(doctor_id)	RESTRICT	CASCADE	BR-23
CONSULTATION_REQUEST	consulting_doctor_id	DOCTOR(doctor_id)	RESTRICT	CASCADE	BR-21/22
PRESCRIPTION	record_id	MEDICAL_RECORD(record_id)	RESTRICT	CASCADE	BR-14
PRESCRIPTION_ITEM	prescription_id	PRESCRIPTION(prescription_id)	CASCADE	CASCADE	BR-16
PRESCRIPTION_ITEM	medication_id	MEDICATION(medication_id)	RESTRICT	CASCADE	BR-16
MEDICATION_AUDIT_LOG	medication_id	MEDICATION(medication_id)	RESTRICT	CASCADE	BR-25
MEDICATION_AUDIT_LOG	admin_id	ADMIN(admin_id)	RESTRICT	CASCADE	BR-25
MEDICAL_RECORD_AUDIT_LOG	record_id	MEDICAL_RECORD(record_id)	RESTRICT	CASCADE	NFR-04
MEDICAL_RECORD_AUDIT_LOG	patient_id	PATIENT(patient_id)	RESTRICT	CASCADE	NFR-04
MEDICAL_RECORD_AUDIT_LOG	doctor_id	DOCTOR(doctor_id)	RESTRICT	CASCADE	NFR-04
MEDICAL_RECORD_AUDIT_LOG	admin_id	ADMIN(admin_id)	RESTRICT	CASCADE	NFR-04

2.3 Normalization (3NF / BCNF Proof)

Every table is verified against 1NF, 2NF, 3NF, and BCNF standards using functional dependencies (FDs) derived from system business rules:

Table	Functional Dependencies	Normalization Verdict
ADMIN	admin_id → full_name, role	1NF: atomic attrs. 2NF: single-attribute PK, no partial dependency possible. 3NF/BCNF: full_name and role depend only on admin_id, no transitive dependency. Holds BCNF.
STAFF_PERMISSION	permission_id → staff_id, access_level, granted_by, granted_at	1NF/2NF: single PK, atomic attrs. 3NF: all attrs depend directly on permission_id; neither staff_id nor granted_by determines any other attribute. Holds BCNF.
DOCTOR	doctor_id → full_name, license_no, spec_type, employment_type, created_by	1NF/2NF: single PK. 3NF: license_no is a candidate key (UNIQUE) but not used as PK; no attribute depends on license_no instead of doctor_id, so no transitive dependency. Holds BCNF.
GP	doctor_id → (no non-key attribute)	Trivial: PK = FK, no other attributes, so no non-trivial FD exists. Holds BCNF by definition.
SPECIALIST	doctor_id → specialty, years_of_practice	1NF/2NF/3NF: single PK, both attrs depend only on doctor_id; specialty does not determine years_of_practice. Holds BCNF.
ROOM	room_id → room_number, floor	1NF/2NF/3NF: single PK, no transitive dependency (floor does not determine room_number). Holds BCNF.

Table	Functional Dependencies	Normalization Verdict
SHIFT	shift_id → doctor_id, room_id, shift_date, start_time, end_time, rate_per_hour	1NF/2NF: single PK. 3NF: rate_per_hour is the rate agreed for that individual shift, fixed at scheduling time from the doctor's seniority and the shift type (evening/weekend rates differ), per BR-24. It is therefore not functionally determined by doctor_id alone, and the chain shift_id → doctor_id → rate_per_hour does not hold. Storing it on SHIFT also preserves the historical rate of completed shifts when the rate card is later revised. Holds BCNF.
PATIENT	patient_id → full_name, dob, phone, email, address	1NF/2NF/3NF: single PK, all attrs describe the patient directly. BR-00 (phone or email must be present) is a tuple-level CHECK constraint, not an FD, and does not affect normalization. Holds BCNF.
APPOINTMENT	appointment_id → patient_id, doctor_id, shift_id, appt_datetime, appt_type, status	1NF/2NF: single PK. 3NF check: doctor_id is technically derivable via shift_id → SHIFT.doctor_id, which looks like a transitive dependency (appointment_id → shift_id → doctor_id). RESOLVED: doctor_id is kept as a direct FK by design decision (BR-07) to allow querying a doctor's appointments without joining through SHIFT, and it is enforced to always match SHIFT.doctor_id via a CHECK/trigger in Phase 3. This is a deliberate, documented redundancy for query performance (NFR-03), not an undetected anomaly. Functionally in BCNF once the doctor_id/shift_id consistency constraint is enforced.
MEDICAL_RECORD	record_id → appointment_id, vital_signs, symptoms, diagnosis, notes	1NF/2NF/3NF: single PK, appointment_id is UNIQUE (1:0..1), all other attrs depend only on record_id. Holds BCNF.
CONSULTATION_REQUEST	consultation_id → record_id, requesting_doctor_id, consulting_doctor_id, reason, status, requested_at, responded_at	1NF/2NF/3NF: single PK, all attrs depend only on consultation_id. The two FKs to DOCTOR play different roles (requesting vs consulting) and neither determines the other, so no redundancy. Holds BCNF.
PRESCRIPTION	prescription_id → record_id, issued_date	1NF/2NF/3NF: single PK, record_id is UNIQUE (1:0..1). Holds BCNF.
PRESCRIPTION_ITEM	(prescription_id, medication_id) → quantity, dosage, frequency, duration_days	1NF: atomic attrs. 2NF: composite PK — quantity/dosage/frequency/duration_days depend on the full pair, not on prescription_id or medication_id alone (dosage for aspirin differs per prescription). No partial dependency. 3NF/BCNF: no transitive dependency among non-key attrs. Holds BCNF.
MEDICATION	medication_id → name, description, is_active	1NF/2NF/3NF: single PK, all attrs depend only on medication_id. name is deliberately not UNIQUE, since different manufacturers may market products under the same commercial name. Holds BCNF.
MEDICATION_AUDIT_LOG	log_id → medication_id, admin_id, action, action_at	1NF/2NF/3NF: single PK, all attrs depend only on log_id (this is an append-only log, not aggregated data). Holds BCNF.
MEDICAL_RECORD_AUDIT_LOG	audit_id → record_id, patient_id, doctor_id, admin_id, action, action_at	1NF/2NF: single PK and atomic attributes. 3NF: all non-key attributes depend directly on audit_id; the actor columns are mutually exclusive by a CHECK constraint (exactly one actor). Append-only audit entries preserve the historical access trail. Holds BCNF.

3. Data Dictionary (Adapted from ISO/IEC 11179)

ADMIN

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
admin_id	INT	PK	No	None	Unique system identifier for admin.
full_name	VARCHAR(100)	None	No	None	Full legal name of administrator.
role	VARCHAR(20)	None	No	None	CHECK (role IN ('RECEPTIONIST', 'SYSTEM_ADMIN')) (BR-18).

STAFF_PERMISSION

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
permission_id	INT	PK	No	None	Unique identifier for permission record.
staff_id	INT	FK	No	None	References ADMIN(admin_id) (BR-20).
access_level	VARCHAR(50)	None	No	None	Granular permission name (BR-20).
granted_by	INT	FK	No	None	References ADMIN(admin_id) (BR-20).
granted_at	DATETIME	None	No	CURRENT_TIMESTAMP	Timestamp recorded when the permission assignment is granted (BR-20).

DOCTOR

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
doctor_id	INT	PK	No	None	Unique identifier for doctor.
full_name	VARCHAR(100)	None	No	None	Full legal name of doctor.
license_no	VARCHAR(50)	None	No	None	UNIQUE; official medical license number.
spec_type	VARCHAR(20)	None	No	None	CHECK (spec_type IN ('GP', 'SPECIALIST')) (BR-01).
employment_type	VARCHAR(20)	None	No	None	CHECK (employment_type IN ('FULL_TIME', 'PART_TIME')) (BR-24).
created_by	INT	FK	No	None	References ADMIN(admin_id) (BR-18).

GP

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
doctor_id	INT	PK,FK	No	None	References DOCTOR(doctor_id) (BR-01).

SPECIALIST

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
doctor_id	INT	PK, FK	No	None	References DOCTOR(doctor_id) (BR-01).
specialty	VARCHAR(50)	None	No	None	Clinical specialty area (BR-02).
years_of_practice	INT	None	No	None	CHECK (years_of_practice >= 0) (BR-02).

ROOM

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
room_id	INT	PK	No	None	Unique room identifier.
room_number	VARCHAR(10)	None	No	None	Room physical identifier (BR-03).
floor	INT	None	Yes	NULL	Floor level of consultation room.

SHIFT

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
shift_id	INT	PK	No	None	Unique shift identifier.
doctor_id	INT	FK	No	None	References DOCTOR(doctor_id) (BR-04).
room_id	INT	FK	Yes	NULL	References ROOM(room_id); mandatory for GP shifts only (BR-03).
shift_date	DATE	None	No	None	Date of scheduled shift (BR-05).
start_time	TIME	None	No	None	Working block start time.
end_time	TIME	None	No	None	Working block end time (BR-05).
rate_per_hour	DECIMAL(10,2)	None	Yes	NULL	Mandatory & > 0 if doctor is PART_TIME, else NULL (BR-24).

PATIENT

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
patient_id	INT	PK	No	None	Unique identifier for patient profile.
full_name	VARCHAR(100)	None	No	None	Patient's full name.
dob	DATE	None	Yes	NULL	Patient date of birth.
phone	VARCHAR(20)	None	Yes	NULL	CHECK (phone IS NOT NULL OR email IS NOT NULL) (BR-00).
email	VARCHAR(100)	None	Yes	NULL	CHECK (phone IS NOT NULL OR email IS NOT NULL) (BR-00).
address	VARCHAR(255)	None	Yes	NULL	Physical home address.

APPOINTMENT

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
appointment_id	INT	PK	No	None	Unique appointment identifier.
patient_id	INT	FK	No	None	References PATIENT(patient_id) (BR-06).
doctor_id	INT	FK	No	None	References DOCTOR(doctor_id) (BR-07).
shift_id	INT	FK	No	None	References SHIFT(shift_id) (BR-08).
appt_datetime	DATETIME	None	No	None	Timestamp of booking (BR-08, BR-09).
appt_type	VARCHAR(20)	None	No	None	CHECK (appt_type IN ('IN_PERSON', 'TELEMEDICINE')) (BR-10).
status	VARCHAR(20)	None	No	'PENDING'	CHECK (status IN ('PENDING', 'CONFIRMED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED')) (BR-11).

MEDICAL_RECORD

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
record_id	INT	PK	No	None	Unique medical record identifier.
appointment_id	INT	FK	No	None	UNIQUE; references APPOINTMENT(appointment_id) (BR-12).
vital_signs	TEXT	None	Yes	NULL	Encrypted clinical telemetry (AES-256) (NFR-04).
symptoms	TEXT	None	Yes	NULL	Patient-reported symptoms.
diagnosis	TEXT	None	Yes	NULL	Encrypted clinical diagnosis (AES-256) (NFR-04).
notes	TEXT	None	Yes	NULL	Attending doctor's clinical notes (BR-13).

CONSULTATION_REQUEST

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
consultation_id	INT	PK	No	None	Unique consultation identifier.
record_id	INT	FK	No	None	References MEDICAL_RECORD(record_id) (BR-22).
requesting_doctor_id	INT	FK	No	None	References DOCTOR(doctor_id) (BR-23).
consulting_doctor_id	INT	FK	No	None	References DOCTOR(doctor_id) (BR-21, BR-23).
reason	TEXT	None	No	None	Medical reason for second opinion referral.
status	VARCHAR(20)	None	No	'PENDING'	CHECK (status IN ('PENDING', 'APPROVED', 'DECLINED', 'COMPLETED')) (BR-23).

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
requested_at	DATETIME	None	No	CURRENT_TIMESTAMP	Referral submission timestamp.
responded_at	DATETIME	None	Yes	NULL	Timestamp of response from consulting doctor.

PRESCRIPTION

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
prescription_id	INT	PK	No	None	Unique prescription identifier.
record_id	INT	FK	No	None	UNIQUE; references MEDICAL_RECORD(record_id) (BR-14).
issued_date	DATE	None	No	CURRENT_DATE	Prescription issuance date.

PRESCRIPTION_ITEM

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
prescription_id	INT	PK, FK	No	None	References PRESCRIPTION(prescription_id) (BR-16).
medication_id	INT	PK, FK	No	None	References MEDICATION(medication_id) (BR-16).
quantity	INT	None	No	None	CHECK (quantity > 0) (BR-17). Encrypted at rest (AES-256) as required by NFR-04.
dosage	VARCHAR(100)	None	No	None	Dosage regimen instructions (BR-17). Encrypted at rest (AES-256) as required by NFR-04.
frequency	VARCHAR(50)	None	No	None	Frequency schedule instructions (BR-17). Encrypted at rest (AES-256) as required by NFR-04.
duration_days	INT	None	No	None	CHECK (duration_days > 0) (BR-17). Encrypted at rest (AES-256) as required by NFR-04.

MEDICATION

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
medication_id	INT	PK	No	None	Unique medication identifier.
name	VARCHAR(100)	None	No	None	Commercial or generic drug name.
description	TEXT	None	Yes	NULL	Drug description and indications.
is_active	BOOLEAN	None	No	TRUE	Logical deactivation flag; prevents hard deletion (BR-19).

MEDICATION_AUDIT_LOG

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
log_id	INT	PK	No	None	Unique audit entry identifier.
medication_id	INT	FK	No	None	References MEDICATION(medication_id) (BR-25).
admin_id	INT	FK	No	None	References ADMIN(admin_id) (BR-25).
action	VARCHAR(20)	None	No	None	CHECK (action IN ('CREATE', 'UPDATE', 'DEACTIVATE')) (BR-25).
action_at	DATETIME	None	No	CURRENT_TIMESTAMP	Immutable timestamp of administrative action (BR-25).

MEDICAL_RECORD_AUDIT_LOG

Attribute Name	Data Type	Key Type	Nullable?	Default Value	Business Rules / Constraints
audit_id	INT	PK	No	None	Unique audit entry identifier (NFR-04).
record_id	INT	FK	No	None	References MEDICAL_RECORD(record_id); historical audit entry cannot exist without its target record (NFR-04).
patient_id	INT	FK	Yes	NULL	References PATIENT(patient_id); exactly one of patient_id, doctor_id, admin_id must be non-NULL to identify the acting user (NFR-04).
doctor_id	INT	FK	Yes	NULL	References DOCTOR(doctor_id); exactly one actor FK is non-NULL (NFR-04).
admin_id	INT	FK	Yes	NULL	References ADMIN(admin_id); exactly one actor FK is non-NULL (NFR-04).
action	VARCHAR(10)	None	No	None	CHECK (action IN ('SELECT', 'INSERT', 'UPDATE')); immutable audit event (NFR-04).
action_at	DATETIME	None	No	CURRENT_TIMESTAMP	Timestamp of the audited operation (NFR-04).